

Тема 8. ГРАФИЧНИ ПРИМИТИВИ

Алгоритми за графично представяне на примитивите окръжности, елипси, правоъгълници. Работа с графични примитиви в средата на OpenGL.

упр. 10 OpenGL Работа с графичните примитиви точки, линии, криви, стандартни графични обекти

ИЗЧЕРТАВАНЕ НА КРИВИ ЛИНИИ. ОБЩИ ПОЛОЖЕНИЯ

Кривите линии се строят с помощта на стандартни методи, а именно апроксимация между точки, получени като резултат от някакъв **експеримент** или от **уравнението на кривата**, ако то е известно. Участъците от кривата линия може да се апроксимират с отсечки от прави или близко разположени точки. Колкото по-голяма е разделителната способност на екрана, толкова по-точна е апроксимацията, в случай, че е известно уравнението на кривата. В случай, че точките са от експеримент, то тогава качеството на графиката и кривите, свързващи точките, зависи и от методите на интерполация, които ще се използват. Колкото повече са апроксимиращите отсечки, заключени между две точки на дъгите (x_1, y_1) и (x_2, y_2) , толкова кривата става по-гладка. Всъщност тя винаги се състои от прави отсечки, но когато те са повече, се създава илюзия за крива. От друга страна, обаче, когато точките са повече, процесът на изчисляването им е по-дълъг и понякога се налага да се ограничи броя на точките, за да се ускори процесът за сметка на качеството на изображението.

8.1. АЛГОРИТМИ ЗА ИЗЧЕРТАВАНЕ НА ОКРЪЖНОСТ

Това е най-често използваната крива. Тя е основен компонент за изчертаване на рисунки, кръгови диаграми и много други сложни графични изображения.

За построяване на окръжност на екрана е необходимо да се укажат **координатите на центъра и радиуса r** . Необходимо е да се определят координатите на точките от окръжността.

Уравнението на окръжността може да се запише по няколко начина. Координатите на последователни точки (x, y) , принадлежащи на окръжността, удобно се определят в зависимост от ъгъла α , измерван по часовниковата стрелка от хоризонталната линия:

$$\begin{aligned}x &= x_C + r \cdot \cos(\alpha) \\ y &= y_C + r \cdot \sin(\alpha)\end{aligned}$$

8.1.1. Алгоритъм с апроксимация на отрязъците [9]

Въвеждат се стойностите на x_C, y_C, r и броя на апроксимиращите отсечки n .

За $\alpha=0$ до 2π със стъпка $2\pi / n$ се пресмятат x и y .

Чертая се права от последната точка до новоизчислената.

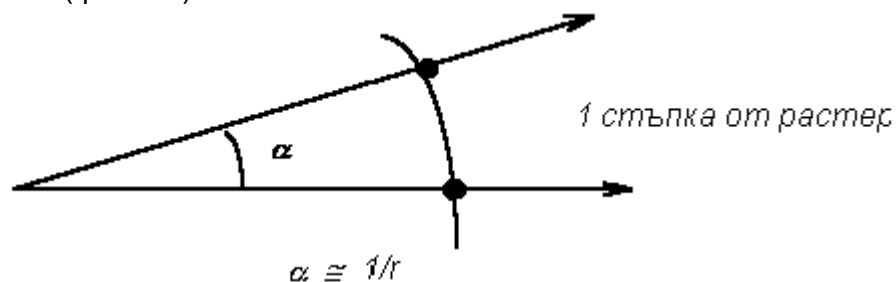
Намаляването на броя на апроксимиращите отсечки увеличава скоростта на изобразяване на екрана. Обаче при това могат да станат забележими отделните отсечки и вместо окръжност ще се получи многоъгълник.

В системи с различна разделителна способност по осите x и y вместо окръжност ще бъде построена елипса. Разликата в разделителната способност може да се компенсира чрез коригиране на стойностите на y , като $r \cdot \sin(\alpha)$ се умножи с отношението на вертикалната разделителна способност на хоризонталната.

8.1.2. Алгоритъм с апроксимация на пикселите [14]

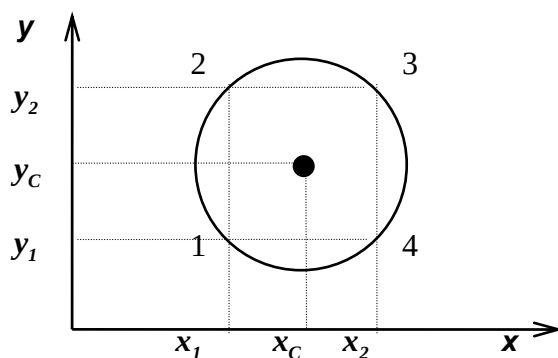
Скоростта на изчертаването на окръжност може да се увеличи, като се засветят пиксели, без да се чертаят съединителните линии. Колкото по-голям е броят на пикселите, толкова по-добре се апроксимира окръжността. Най-добър резултат се постига, когато разстоянието

между съседни пиксели е равно на една стъпка по растера по вертикалата и хоризонталата. Ъгловото разстояние (в радиани) е приблизително равно на реципрочната стойност на радиуса на окръжността. По-малката стъпка на изменение на ъгъла ще гарантира отсъствие на промеждутъци между пикселите, но ще доведе до излишни изчисления и повторно засветяване на точки (фиг. 7.4).



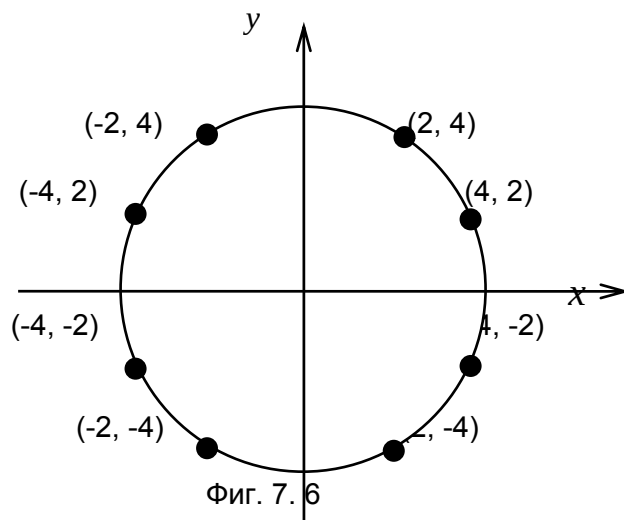
Фиг. 7.4

Възможно е и да се отчете симетрията на окръжността и да се изчисляват координатите само на някои пиксели. Горната половина на окръжността е симетрична на долната, лявата на дясната. Това значи, че на всяка стойност на x съответстват две стойности на y , а на всяка стойност на y съответстват две такива на x (фиг. 7.5).



Фиг. 7.5

Освен това, може да се получат още 4 пиксела от окръжността, изменяйки местата на координатите x и y , т.е. ако точка (x, y) принадлежи на окръжността, то точка (y, x) също ще ѝ принадлежи (фиг. 7.6). Така става необходимо да се изчислят координатите на точките само на $1/8$ част от окръжността, а от тях се получават всички останали пиксели.



Фиг. 7.6

Времето за изпълнение може да се намали, ако се изчисляват координатите на точките без използване на функциите $\sin$ и $\cos$. Този метод е основан на изчисляване на координатите на поредната точка по координатите на предишната точка. Ако точката (x_1, y_1) принадлежи на окръжността, координатите на следващата точка (x_2, y_2) могат да се изчислят по формулата:

$$x_2 = x_C + (x_1 - x_C).ca + (y_1 - y_C).sa \quad (7.2)$$

$$y_2 = y_C + (y_1 - y_C).ca - (x_1 - x_C).sa$$

Тук ca и sa са константи, чиито стойности зависят от стъпката на изменение на ъгъла da и са равни на:

$$ca = \cos(da)$$

$$sa = \sin(da)$$

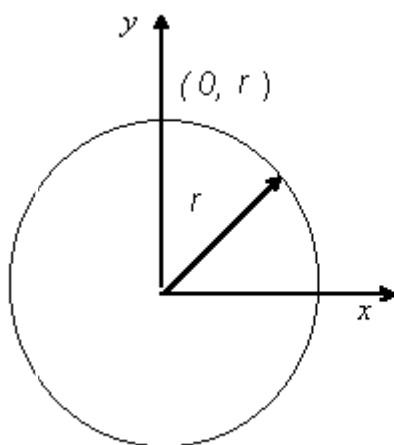
Началната точка е с координати $x_1 = x_C$, $y_1 = y_C + r$. Изчисленията се прекратяват, когато $x_2 - x_C > y_2 - y_C$.

Използвайки този метод, можем да получим 8 точки от окръжността за всяка точка, изчислена в съответствие с уравнения (7.2).

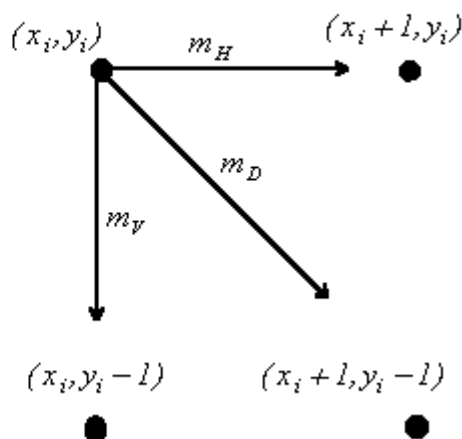
8.1.3. Алгоритъм на Брезенхайм за генерация на окръжност

Един от най-ефективните и лесни за разбиране алгоритми за генерация на окръжност принадлежи на Брезенхайм [11]. Необходимо е да се генерира само една осма част от окръжността, а останалите се получават като последователни отражения.

Разглеждаме първата четвърт от окръжността с център в началото на координатната система. Ако работата на алгоритъма започва в точка $x=0, y=r$, то при генерирането на окръжността в първи квадрант y се явява монотонно намаляваща функция на аргумента x (фиг. 7.7). Аналогично, ако изходна точка е $y=0, x=r$, то при генерация обратно на часовата стрелка x ще бъде монотонно намаляваща функция на аргумента y . В случая е избрана генерация по часовата стрелка с начало в точката $x=0, y=r$.



Фиг. 7.7



Фиг. 7.8

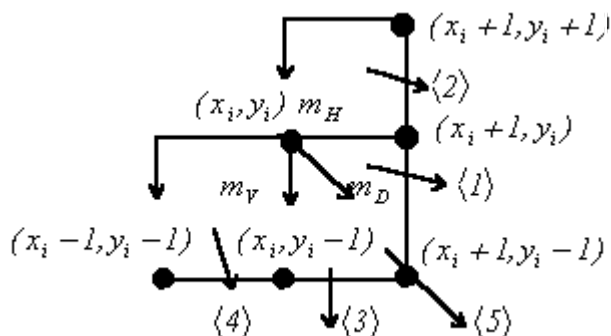
За всяка зададена точка съществуват само три възможности да се избере следващия пиксел, намиращ се най-близо до окръжността: хоризонтално надясно, по диагонал надолу и надясно, вертикално надолу (фиг. 7.8). Алгоритъмът избира пиксел, за който е минимален квадрата на разстоянието между един от тези пиксели и окръжността, т.е. минималното от:

$$m_H = |(x_i + 1)^2 + (y_i)^2 - r^2|$$

$$m_D = |(x_i + 1)^2 + (y_i - 1)^2 - r^2|$$

$$m_V = |(x_i)^2 + (y_i - 1)^2 - r^2|$$

В околността на точка (x_i, y_i) са възможни само пет типа пресичания на окръжността и мрежата на растера :



Фиг. 7.9

Разликата между квадратите на разстоянията от центъра на окръжността до диагонален пиксел $(x_i + 1, y_i - 1)$ и от центъра до точката на окръжността r е:

$$\Delta_i = \left| (x_i + 1)^2 + (y_i - 1)^2 - r^2 \right|$$

Както и в алгоритъма на Брезанхайм за генериране на отсечка, за избор на съответния пиксел е желателно да се използва само знакът на грешката, а не нейната големина.

Може да се докаже, че при :

$$\Delta_i < 0$$

$\delta \leq 0$ избираме пиксел $(x_i + 1, y_i) \rightarrow m_H$

$\delta > 0$ избираме пиксел $(x_i + 1, y_i - 1) \rightarrow m_D$

$$\text{където } \delta = m_H - m_D = \left| (x_i + 1)^2 + (y_i)^2 - r^2 \right| - \left| (x_i + 1)^2 + (y_i - 1)^2 - r^2 \right|$$

$$\Delta_i > 0$$

$\delta \leq 0$ избираме пиксел $(x_i + 1, y_i - 1) \rightarrow m_D$

$\delta > 0$ избираме пиксел $(x_i, y_i - 1) \rightarrow m_V$

$$\text{където } \delta = m_D - m_V = \left| (x_i + 1)^2 + (y_i - 1)^2 - r^2 \right| - \left| (x_i)^2 + (y_i - 1)^2 - r^2 \right|$$

$\Delta_i = 0$ избираме пиксел $(x_i + 1, y_i - 1) \rightarrow m_D$

Примерна реализация на Постъпков алгоритъм на Брезанхайм за генериране на окръжност в първи квадрант [11]

всички променливи са цели
инициализация на променливите

$$x_i = 0$$

$$y_i = r$$

$$\Delta_i = 2 \cdot (1 - r)$$

$$Granica = 0$$

1 Plot (x_i, y_i)

if $y_i \leq Granica$ then 4 endif

Отделяне на случай 1 или 2, 4 или 5, или 3

if $\Delta_i < 0$ then 2 endif

```

    if  $\Delta_i > 0$  then 3 endif
    if  $\Delta_i = 0$  then 20 endif
Определяне на случай 1 или 2
2     $\delta = 2\Delta_i + 2y_i - 1$ 
    if  $\delta \leq 0$  then 10 endif
    if  $\delta > 0$  then 20 endif
Определяне на случай 4 или 5
3     $\delta = 2\Delta_i + 2x_i - 1$ 
    if  $\delta \leq 0$  then 20 endif
    if  $\delta > 0$  then 30 endif
Изпълнение на стъпките
    стъпка  $m_H$ 
10    $x_i = x_i + 1$ 
       $\Delta_i = \Delta_i + 2x_i + 1$ 
      go to 1
    стъпка  $m_D$ 
20    $x_i = x_i + 1$ 
       $y_i = y_i - 1$ 
       $\Delta_i = \Delta_i + 2x_i - 2y_i + 2$ 
    стъпка  $m_V$ 
30    $y_i = y_i - 1$ 
       $\Delta_i = \Delta_i - 2y_i + 1$ 
      go to 1
finish

```

Променливата *Granica* става равна на 0 при завършването на работата на алгоритъма на хоризонталната ос, като в резултат се генерира окръжност в първи квадрант. Ако е необходимо да се получи само един октант, то втория може да се получи чрез установяване на $Granica = Integer(r/\sqrt{2})$ а първия - чрез отражение на втория октант относно правата $y=x$.

Алгоритъмът може леко да се обобщи за другите квадранти или за дъга от окръжност.

8.2 OPENGL - ПРОСТИ ОБЕКТИ

OpenGL предоставя функции за изрисване само на прости обекти - линии, триъгълници, полигони, които могат да бъдат свързани в някоя по-сложна фигура. Разбира се и библиотеките **GLAUX** и **GLUT** правят лесно създаването на често използвани фигури като сфера, куб и различни многоъгълници. Библиотеката **GLU** дори представя функции, с които може да създаде иначе доста сложни обекти като цилиндър, диск, криви, релеф.

За да се създаде проста фигура трябва да дефинирате нейните точки (т.е. нейните върхове). За всеки един връх може да се определи цвят, полу-прозрачност и нормала (вектор, който се използва за определянето на динамичната светлина, която попада върху съответната повърхност). Определянето на върховете на даден обект трябва да стои между функциите **glBegin(GLenum Mode)** и **glEnd()**. Те показват началото и края на дадено изрисване. За всеки връх се използва текущия цвят, който може да се промени при повторно извикване на определена функция но с различни аргументи.

Определянето на координатите на точка става с функцията :

glVertex{234}fv(x, y, z, (w)) - функцията може да приема от два до четири аргумента, а също и масив от стойности. За определяне на някаква точка в **3D** пространството се извиква : **glVertex3f(x, y, z)** - аргументът w е тегловния коефициент, (по подразбиране равен на 1) и на него се делят еднородните координати x, y и z.

Определянето на цвят в **RGBA** режим става с функцията:

glColor{34}fv(R, G, B, (A)) - цвета се изгражда в зависимост от наситеността на червения, синия и зеления цвят, която се определя в части от цялото [0;1]. Ако искате наполовина

наситено червено трябва да извикате функцията със следните параметри : **glColor3f(0.5 , 0.0 , 0.0);**

Четвъртият аргумент, който не е задължителен е прозрачността. Тя обаче трябва да се зададе предварително с други функции. Полу-прозрачността се разглежда в друга тема. Стойността 1.0 задава че няма полу-прозрачност.

При **Color-Index** режим функцията за определяне на цвят е :

glIndex(index) - като число от 0 - 255 определя различен цвят.

Може да се определи различен цвят за всеки връх и така фигурата ще прелива в зависимост от цвета на нейните върхове.

С функцията **glShadeModel(GLenum mode)** се определя начина на определяне на цвета. Тя може да приеме една от константите:

GL_SMOOTH - за преливащ цвят, което е по-подразбиране.

GL_FLAT - за едноцветни фигури, независимо от цвета на върховете.

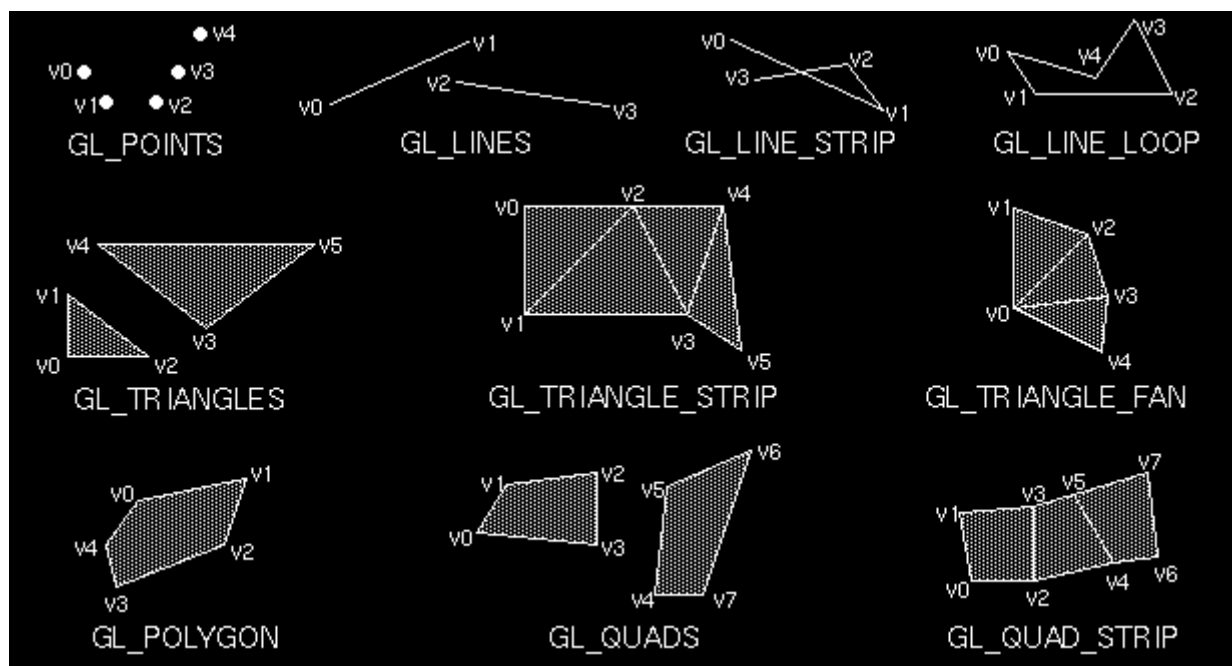
Преди да изрисувате някакъв обект трябва да се подадат като аргумент на функцията **glBegin(GLenum mode)** една от следните константи:

GL_POINTS - остават отделни точки.

GL_LINES - точките се свързват две по две в отделни линии. Ако броя на точките е нечетен, последната точка остава несвързана.

GL_LINE_STRIP - всяка точка се свързва със следващата.

GL_LINE_LOOP - всяка точка се свързва със следващата, като последната се свързва с първата.



GL_TRIANGLES - точките се свързват в отделни триъгълници. Ако останат една или две точки те не се свързват.

GL_TRIANGLE_STRIP - точките се свързват в серия от триъгълници. Първо се свързват точки 1, 2 и 3, след това 3, 2 и 4, следват 3, 4 и 5, след това 4, 5 и 6 и т.н.

GL_TRIANGLE_FAN - точките се свързват в отново в серия от триъгълници, но начина на свързване е различен. Първо 1, 2 и 3 после 1, 3 и 4 след това 1, 4 и 5 и т.н.

Така се получава някакво подобие на диск защото всички триъгълници имат една допирна точка.

GL_QUADS - точките се свързват в отделни четириъгълници. Ако броя на точките не е кратен на 4, последните остават несвързани.

GL_QUAD_STRIP - точките се свързват в серия от четириъгълници, като първо се свързват точки 1, 2, 4 и 3 следват 3, 4, 6 и 5, след това 5, 6, 8 и 7 и т.н.

GL_POLYGON - точките се свързват в полигон, който трябва да е задължително изпъкнал, иначе са възможни аномалии.

Функции от високо ниво, които създават наготово някои по-сложни обекти :

glRectf(x1, x2, y1, y2) - тази функция изгражда правоъгълник. Съответно (x1, y1) са координатите на горния ляв ъгъл, а (x2, y2) на долния десен ъгъл.

Функции за изрисване на тримерни обекти от помощната библиотека GLUT :

glutSolidCube(GLdouble width) - изрисува куб, в зависимост от неговата страна.

glutSolidSphere(GLdouble radius, GLint slices, GLint stacks) - приема радиуса на сферата, но и подразделенията съответно около оста z (меридиани) и по оста z (паралели) т.е. ако се подадат за последните два аргумента числото 20, то сферата ще бъде по-ръбата от сфера с 50 подразделения около и по z координатата, но и по-лека за графичния ускорител.

glutSolidCone(GLdouble base, GLdouble height, GLint slices, GLint stacks) - изрисува конус, като приема радиуса на основата, височината и съответните подразделения като при сферата.

glutSolidTorus(GLdouble innerRadius, GLdouble outerRadius, GLint nsides, GLint rings) - изрисува торус (пръстен) , като приема външния и вътрешния радиус и съответните подразделения.

glutSolidTeapot(GLdouble size) - изрисува чайник, като неговата големина е се генерира в зависимост от подадения аргумент.

Следващите функции не приемат аргумент. По подразбиране техния радиус е равен на корен квадратен от 3. Може да се променят размерите им чрез мащабиране с функцията **glScale()**.

glutSolidDodecahedron(void) - изрисува дванадесетостенник.

glutSolidIcosahedron(void) - изрисува двадесетостенник.

glutSolidOctahedron(void) - изрисува осмостенник.

glutSolidTetrahedron(void) - изрисува четиристенник.

Ако се цели изобразяването на някой от тези обекти само с контурите (линиите), трябва да се използват същите функции, но вместо **Solid** е нужно да се сложи **Wire** в името на функцията. Например:

glutWireCube(GLdouble width).

В **GLUAX** се съдържат същите функции, само че вместо с **glut** започват с **aux**. Разликата се състои в това че повечето от тях приемат различен брой аргументи и безспорно някои от обектите създадени с **GLUT** могат да бъдат много по-детайлни от същите създадени с **GLAUX**. И макар че е по-удачно да използвате библиотеката **GLUT**.

auxSolidCube(GLdouble width) - изрисува куб, като приема за аргумент неговата страна.

auxSolidBox(GLdouble width, GLdouble height, GLdouble depth) - изрисува "кутия", като приема за аргументи, нейната ширина, височина и дебелина.

auxSolidCone(GLdouble radius, GLdouble height) - изрисува конус, като приема за аргументи неговия радиус и височина.

auxSolidCylinder(GLdouble radius, GLdouble height) - изрисува цилиндър, като приема за аргументи неговия радиус и височина.

auxSolidSphere(GLdouble radius) - изрисува сфера, като приема за аргумент нейния радиус.

auxSolidTorus(GLdouble innerRadius, GLdouble outerRadius) - изрисува фигура, чиято форма най-точно наподобява "геврек или пристен" . Функцията приема съответно външния и вътрешния радиус.

auxSolidTeapot(GLdouble size) - изрисува чайник, като неговата големина се генерира в зависимост от подадения аргумент.

auxSolidDodecahedron(GLdouble radius) - изрисува дванадесетостенник.

auxSolidIcosahedron(GLdouble radius) - изрисува двадесетостенник.

auxSolidOctahedron(GLdouble radius) - изрисува осмостенник.

auxSolidTetrahedron(GLdouble radius) - изрисува четиристенник.

Както при **GLUT** и тук можете да определите "линеарно" изрисване с **Wire** вместо **Solid**.

Функции от библиотеките **GLAUX** и **GLUT** не само създават обект, но и определят неговите нормални вектори, нужни за правилното функциониране на динамичната светлина, която пак наистина ще накара триизмерните обекти да изглеждат триизмерно.

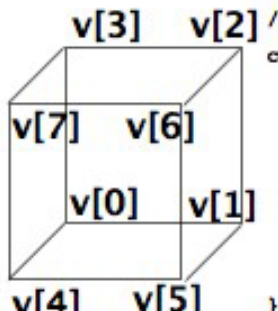
glPolygonMode(face , mode) - тя се отнася за всички видове фигури и определя как да бъдат изрисувани. Първият аргумент може да бъде една от константите **GL_FRONT**, **GL_BACK**, **GL_FRONT_AND_BACK** - съответно за предна страна, задна или и за двете страни. Вторият аргумент може да бъде **GL_FILL** - за пълни фигури, което е и по подразбиране, **GL_POINT** - изобразяват се само върховете или **GL_LINE** - изобразяват се само линиите на обектите.

Примери:

```
float v1[3] = {-1.0,-1.0,0.0};
float v2[3] = { 1.0,-1.0,0.0};
float v3[3] = { 1.0, 1.0,0.0};
float v4[3] = {-1.0, 1.0,0.0};
```

<pre>glBegin(GL_POLYGON); glVertex3f(-1.0,-1.0,0.0); glVertex3f(1.0,-1.0,0.0); glVertex3f(1.0,1.0,0.0); glVertex3f(-1.0,1.0,0.0); glEnd();</pre>	<pre>glBegin(GL_POLYGON); glVertex3fv(v1); glVertex3fv(v2); glVertex3fv(v3); glVertex3fv(v4); glEnd();</pre>
--	--

<pre>typedef float Point[3]; Point verts[8] = { {-1.,-1.,-1.}, { 1.,-1.,-1.}, { 1., 1.,-1.}, {-1., 1.,-1.}, {-1.,-1., 1.}, { 1.,-1., 1.}, { 1., 1., 1.}, {-1., 1., 1.}, };</pre>	<pre>face(int a, int b, int c, int d) { glBegin(GL_POLYGON); glVertex3fv(verts[a]); glVertex3fv(verts[b]); glVertex3fv(verts[c]); glVertex3fv(verts[d]); glEnd(); } // Note consistent ccw orientation! cube() { face(0,3,2,1); face(2,3,7,6); face(0,4,7,3); face(1,2,6,5); face(4,5,6,7); face(0,1,5,4); }</pre>
---	---




```

typedef float Point[3];
Point verts[8] = {
    {-1.,-1.,-1.},
    { 1.,-1.,-1.},
    { 1., 1.,-1.},
    {-1., 1.,-1.},
    {-1.,-1., 1.},
    { 1.,-1., 1.},
    { 1., 1., 1.},
    {-1., 1., 1.},
};
int polys[6][4] = {
    {0,3,2,1},
    {2,3,7,6},
    {0,4,7,3},
    {1,2,6,5},
    {4,5,6,7},
    {0,1,5,4}
};

```

```

face(int poly[4]) {
    glBegin(GL_POLYGON);
        glVertex3fv(poly[0]);
        glVertex3fv(poly[1]);
        glVertex3fv(poly[2]);
        glVertex3fv(poly[3]);
    glEnd();
}

cube() {
    for( int i = 0; i < n; i++ )
        face(polys[i]);
}

```